

MovieMatch

A Hybrid Movie Recommendation System using Machine Learning (SVD, KNN) and Generative AI (Google Gemini)

Student Name(s)	Aarav Gupta	Roll Number(s)	2400320100006
Branch / Department	CSE	College Name	ABES ENGINEERING COLLEGE
Project Guide	Ms. Anshika Chaudhary	Academic Year	2026-27

1. Abstract

This document specifies the requirements for **MovieMatch**, a web-based hybrid movie recommender. The product predicts ratings with scikit-surprise models (**SVD**, item-based **KNN** with Pearson baseline similarity, and BaselineOnly), serves them from a **FastAPI** service over a 3NF **SQLite** database of 80,000+ movies, and presents them in a **React** single-page application. On top of the rating models it adds a Date Night Matchmaker for two users, franchise grouping, and LLM-based features (assistant, synopses, welcome message) reached through OpenRouter using **Gemini 2.5 Flash**. Models are trained offline on a 64,000+ interaction matrix and benchmarked on MovieLens (610 users, 9,742 movies, 100,836 ratings). The sections below cover the problem, requirements, architecture, design, data model and test plan.

2. Problem Statement

Users facing thousands of titles suffer decision fatigue. Existing rating-based lists are opaque, serve one person at a time, and are limited to titles seen during training. The system must therefore combine ML ranking, two-user matching and LLM-driven conversational discovery over a large catalogue.

3. Objectives

- Rank unseen movies for each user with SVD, KNN and Baseline models.
- Support two-user recommendations through a harmonic-mean Date Night Matchmaker.
- Offer LLM-based assistance: chat, dynamic synopses and personalised greetings.
- Maintain an 80,000+ movie catalogue through an ETL pipeline with cached artwork.
- Separate presentation, logic and storage (React / FastAPI / SQL) behind a validated REST API.
- Keep the interface fast, responsive and visually immersive.

4. Proposed Solution

Interaction: the React client sends HTTP/JSON requests to FastAPI. Each request is validated by Pydantic, handled with an injected database session, scored by pre-loaded Pickle models and, only when required, enriched by the LLM. **Resilience:** if OpenRouter or OMDb is unavailable, the system keeps returning ML-based results and shows a gradient placeholder for posters. **Users:** movie viewers, casual streamers and pairs choosing together.

5. Scope of the Project

In scope: Top Picks (SVD, KNN, Baseline), Date Night Matchmaker, franchise rails, LLM assistant, synopses and welcome message, 80,000+ movie ETL with poster caching, ratings and watchlist, validated REST API, 3NF database and the React interface. **Out of scope:** real-time retraining, password login, mobile app, video streaming, social features and deep learning.

6. Functional Requirements

ID	Functional Requirement
FR-01	The system shall let a user log in with a username, rate movies (1–5 stars), keep a Plan to Watch list, and shall track each user's rating history.
FR-02	The system shall generate personalised Top Picks using SVD matrix factorization, with KNN and Baseline available as alternatives.
FR-03	The system shall provide a Date Night Matchmaker that predicts scores for two users across the catalogue and combines them with the harmonic mean $2 \cdot s_1 \cdot s_2 / (s_1 + s_2)$.
FR-04	The system shall provide a floating chat assistant that uses an LLM to interpret natural-language queries and maps the chosen movie to a local database record shown as a clickable card.

ID	Functional Requirement
FR-05	The system shall extract a franchise name from a movie title and list its sequels and prequels in a “The Collection” rail.
FR-06	The system shall generate a two-sentence AI synopsis on demand when a movie has no plot description.
FR-07	On login, the system shall display a personalised AI recommendation based on the user’s top-rated movies.
FR-08	Background daemons shall query OMDb to download and locally cache movie posters .
FR-09	The system shall store users, movies, ratings and watchlist entries in a relational database of 80,000+ movies.
FR-10	The backend shall validate every request with Pydantic and return an HTTP 422 error for invalid input.
FR-11	The frontend shall keep the user’s session in browser localStorage and preserve state while navigating (React Router).

7. Non-Functional Requirements

Performance: endpoints that fetch basic movie data respond in under 200 ms; models load once at start-up and the LLM is called only when needed.

Security: inputs are validated (ratings limited to 1–5), the ORM uses parameterised queries and API keys stay in the backend.

Usability: a dark, cinematic theme inspired by premium streaming platforms, with responsive horizontal-scrolling carousels and readable movie cards.

Reliability: if the OpenRouter LLM API is down, ML-based recommendations continue; if OMDb returns no poster, a fallback gradient box is rendered.

Scalability: the platform must handle and query more than 80,000 movie records; tiers are separate and SQLAlchemy allows moving from SQLite to PostgreSQL.

Compatibility: modern browsers with JavaScript enabled; server needs Python 3.10+ and Node.js; SQLite needs no separate server.

8. Technology Stack

Language	Python 3.10+, JavaScript	Frontend	React, Vite, Tailwind CSS v4, React Router, Lucide-React
Backend	FastAPI, Uvicorn, Pydantic, Dependency Inj.	Database	SQLite via SQLAlchemy 2.1 ORM (3NF)
ML Models	scikit-surprise (SVD, KNNBaseline, Baseline)	Generative AI	LLM (Gemini 2.5 Flash) via OpenRouter API

9. System Architecture

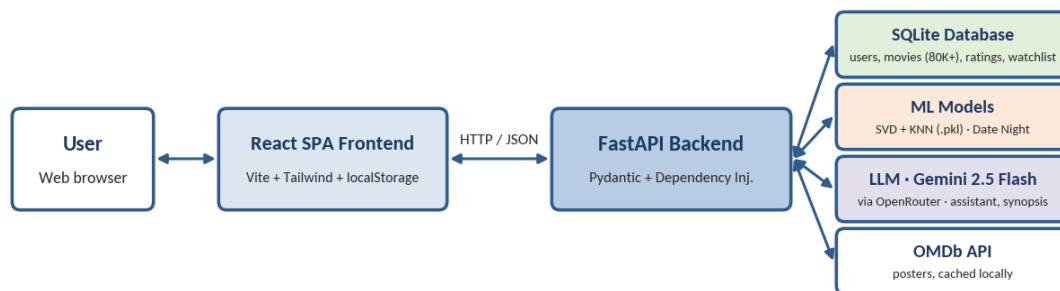


Figure 1: Three-tier architecture – React frontend, FastAPI backend, SQLite database / ML models / LLM (OpenRouter) / OMDb API

10. System Design

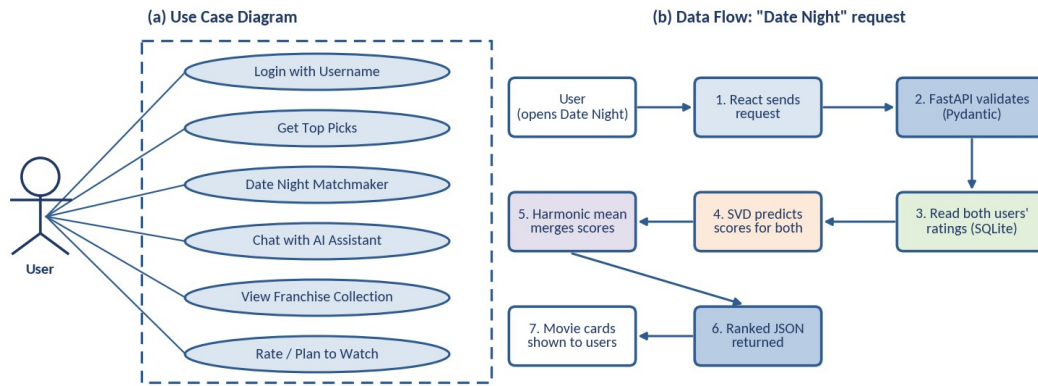


Figure 2: (a) Use case diagram and (b) data flow for a "Date Night" request

11. Module Description

Module	Functions
1. Frontend (React SPA)	Render pages and carousels; call the API; hold the session in localStorage; host the floating assistant.
2. Backend API (FastAPI)	Expose endpoints; validate input (Pydantic); manage DB sessions (dependency injection); coordinate ML, LLM and OMDb.
3. Data Management and ETL	SQLAlchemy models and queries; import 80,000+ movies and clean titles with regex; run poster-caching daemons.
4. Recommendation Engine	Load SVD, KNN and Baseline Pickle files; compute Date Night scores; parse titles for franchise rails.
5. Generative AI	Map chat requests to database records; generate synopses and greetings from structured prompts.

12. Database Design

Table	Key Fields	Purpose
users	user_id (PK), username (unique)	Stores user identity.
movies	movie_id (PK), title, genres	Catalogue of 80,000+ movies used by the models.
ratings	id (PK), user_id, movie_id, rating, timestamp	User ratings (1-5); training data.
watchlist	id (PK), user_id, movie_id, timestamp	Plan to Watch list.

The schema follows **3NF**: no repeated data; tables are linked through user_id and movie_id (SQLAlchemy ORM).

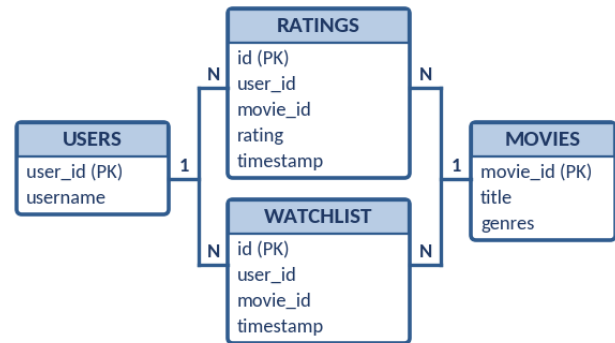


Figure 3: ER diagram

13. Testing

Test ID	Test Case	Expected Result	Status
TC01	Click Top Picks for a user	Personalised, ranked Top Picks are displayed	Pass
TC02	Open a movie with sequels (Toy Story 2)	"The Collection" rail lists the franchise titles	Pending
TC03	Run Date Night for two users	Mutual picks ranked by harmonic mean are displayed	Pending
TC04	Ask the AI assistant for a movie	Matching movie shown as a clickable card	Pending
TC05	Open a movie that has no plot	A two-sentence AI synopsis is displayed	Pass
TC06	Send an invalid API request	Validation error (HTTP 422) is returned	Pass
TC07	Switch off LLM API / missing poster	ML picks continue; fallback poster is shown	Pending

Results: model scores on the MovieLens benchmark – SVD: RMSE 0.879, MAE 0.673, NDCG 0.956; KNN: 0.852, 0.648, 0.963; Baseline: 0.873, 0.671, 0.959.

14. Limitations

- LLM features need internet access and depend on OpenRouter availability, latency and quota.
- Models are trained offline; new ratings are not reflected until retraining.
- The 64,000+ interaction matrix used for SVD is synthetic, so recommendations for new users are less accurate.
- AI-generated text (e.g., synopses, assistant replies) can occasionally be inaccurate.
- SQLite and username-only login limit scale and security.

15. Future Scope

- Automated nightly ML retraining pipeline.
- Neural Collaborative Filtering (deep learning).
- Cloud deployment with a managed PostgreSQL database.
- Caching of LLM responses to cut latency and cost.
- Mobile application and secure password-based accounts.
- AI Director: LLM-curated personalised picks.

16. References

- [1] Y. Koren, R. Bell and C. Volinsky, "Matrix Factorization Techniques for Recommender Systems," IEEE Computer, vol. 42, no. 8, 2009.
- [2] F. Ricci, L. Rokach and B. Shapira (Eds.), Recommender Systems Handbook, Springer.
- [3] F. M. Harper and J. A. Konstan, "The MovieLens Datasets: History and Context," ACM TiIS, vol. 5, no. 4, 2015.
- [4] N. Hug, "Surprise: A Python library for recommender systems," JOSS, vol. 5, no. 52, 2020.
- [5] FastAPI – <https://fastapi.tiangolo.com> | React – <https://react.dev> | Vite – <https://vite.dev>
- [6] Tailwind CSS – <https://tailwindcss.com/docs> | SQLAlchemy – <https://docs.sqlalchemy.org> | Pydantic – <https://docs.pydantic.dev>
- [7] Gemini API – ai.google.dev/gemini-api/docs | OpenRouter – openrouter.ai/docs | OMDb – omdbapi.com
- [8] IMDb Non-Commercial Datasets – <https://developer.imdb.com/non-commercial-datasets/>